torchsimus

Simulador de ultrasonido médico

Instructivo de uso

Versión del paquete: 0.2.0
Repositorio: github.com/GianfrancoOneto/torchsimus

Gianfranco Oneto, Ronal Coronado
Facultad de Ingeniería y Negocios — NIDS
Universidad de las Américas (UDLA)
Octubre 2026

Contenido

1. Qué es torchsimus
- 1.1 El ultrasonido médico en pocas palabras
- 1.2 Cómo se forma una imagen
- 1.3 Para qué sirve simular ultrasonido
- 1.4 Qué hace torchsimus
- 1.5 Flujo de una simulación
- 1.6 Palabras que se usan en este instructivo
- 1.7 Qué necesitas

2. Instalar torchsimus en Google Colab
- 2.1 Actualizar a la última versión
- 2.2 Archivos

3. Primeros pasos: el sonido que emite la sonda
- 3.1 Celda de preparación
- 3.2 Onda enfocada
- 3.3 Onda plana, con y sin atenuación
- 3.4 Onda divergente y varios focos a la vez
- 3.5 Sonda convexa (abdominal)
- 3.6 Cambiar de sonda

4. Simular una imagen de ultrasonido
- 4.1 Imagen modo B de un tejido con un quiste
- 4.2 Película de la propagación del sonido
- 4.3 Procesar datos reales: un disco que gira

5. Diseñar tu propio cabezal
- 5.1 Dos funciones de ayuda
- 5.2 Usar uno de los 35 cabezales de ejemplo
- 5.3 Usar tus propias coordenadas

6. Referencia rápida
- 6.1 Módulos y funciones principales
- 6.2 Sondas incluidas
- 6.3 Unidades y convenciones

7. Confiabilidad y rapidez
- 7.1 Comparación con pymust
- 7.2 Tiempos de cálculo

8. Problemas frecuentes

9. Anexo: el modelo físico

10. Referencias

> **Nota:** si el índice aparece vacío, en Word haz clic derecho sobre él y elige **Actualizar campos**.

1. Qué es torchsimus

torchsimus es un **simulador de ultrasonido médico**. Calcula en un computador cómo viaja el sonido que emite una sonda de ultrasonido, qué ecos vuelven desde el cuerpo y qué se puede obtener a partir de ellos (imágenes, flujo sanguíneo, movimiento de los tejidos), sin necesidad de un equipo real.

1.1 El ultrasonido médico en pocas palabras

El **ultrasonido médico** usa sonido de muy alta frecuencia, imposible de oír: entre 2 y 15 millones de vibraciones por segundo (2 a 15 MHz). No usa radiación y tiene muchos usos: obtener imágenes del interior del cuerpo (la más conocida es la **ecografía**), medir la velocidad de la sangre (**Doppler**), seguir el movimiento de los tejidos y del corazón, y diseñar nuevas sondas y formas de emitir el sonido.

El examen se hace con una **sonda** (también llamada transductor o cabezal) que se apoya sobre la piel. Dentro de la sonda hay una fila o una grilla de pequeños **elementos piezoeléctricos**, que vibran cuando reciben un pulso eléctrico y, al revés, generan una señal eléctrica cuando les llega un eco.

1.2 Cómo se forma una imagen

1. **Emisión.** Los elementos de la sonda emiten un pulso de sonido muy corto. Disparándolos con pequeños retardos entre sí, el sonido se puede **enfocar** en un punto, enviar como **onda plana** o abrir como **onda divergente**.
2. **Propagación.** El sonido avanza por los tejidos a unos 1540 metros por segundo y se va debilitando con la profundidad (**atenuación**). La atenuación es mayor a frecuencias altas.
3. **Ecos.** Cada vez que el sonido encuentra un cambio de tejido o pequeñas estructuras (células, fibras), una parte rebota. Esas pequeñas fuentes de eco se llaman **dispersores** (scatterers).
4. **Recepción.** Los mismos elementos de la sonda registran los ecos que vuelven. Cada elemento entrega una señal llamada **señal RF**.
5. **Formación de la imagen.** Un algoritmo llamado **beamforming** combina las señales de todos los elementos para saber de qué punto del cuerpo vino cada eco. Con eso se arma la imagen en **modo B**: la ecografía en blanco y negro, donde lo que refleja mucho se ve claro y lo que no refleja (por ejemplo, un quiste con líquido) se ve oscuro.

Además de la imagen, los ecos permiten medir **movimiento**. El **Doppler color** muestra la velocidad de la sangre (rojo si se acerca a la sonda, azul si se aleja), y el **speckle tracking** sigue el movimiento de los tejidos de una imagen a la siguiente.

Hay un compromiso básico: a mayor frecuencia, la imagen es más nítida pero el sonido llega menos profundo. Por eso existen sondas distintas: **lineales** (alta frecuencia, para estructuras cercanas a la piel), **convexas** (curvas, para el abdomen), **phased array** (pequeñas, para ver el corazón entre las costillas) y **matriciales** (para imágenes 3-D).

1.3 Para qué sirve simular ultrasonido

Simular ultrasonido médico permite trabajar sin pacientes ni equipos: **diseñar sondas** (transductores) antes de fabricarlas, **probar distintas formas de emitir** el sonido y **generar imágenes de entrenamiento** para algoritmos de diagnóstico por imagen, como redes neuronales.

El simulador de referencia en este campo es **SIMUS**, parte de la herramienta MUST, que fue validado frente a otros simuladores [1–3]. Sin embargo, está escrito en MATLAB, y su versión en Python (pymust) usa NumPy, así que ninguna de las dos puede usar tarjeta gráfica (GPU) ni calcular derivadas. torchsimus resuelve eso.

1.4 Qué hace torchsimus

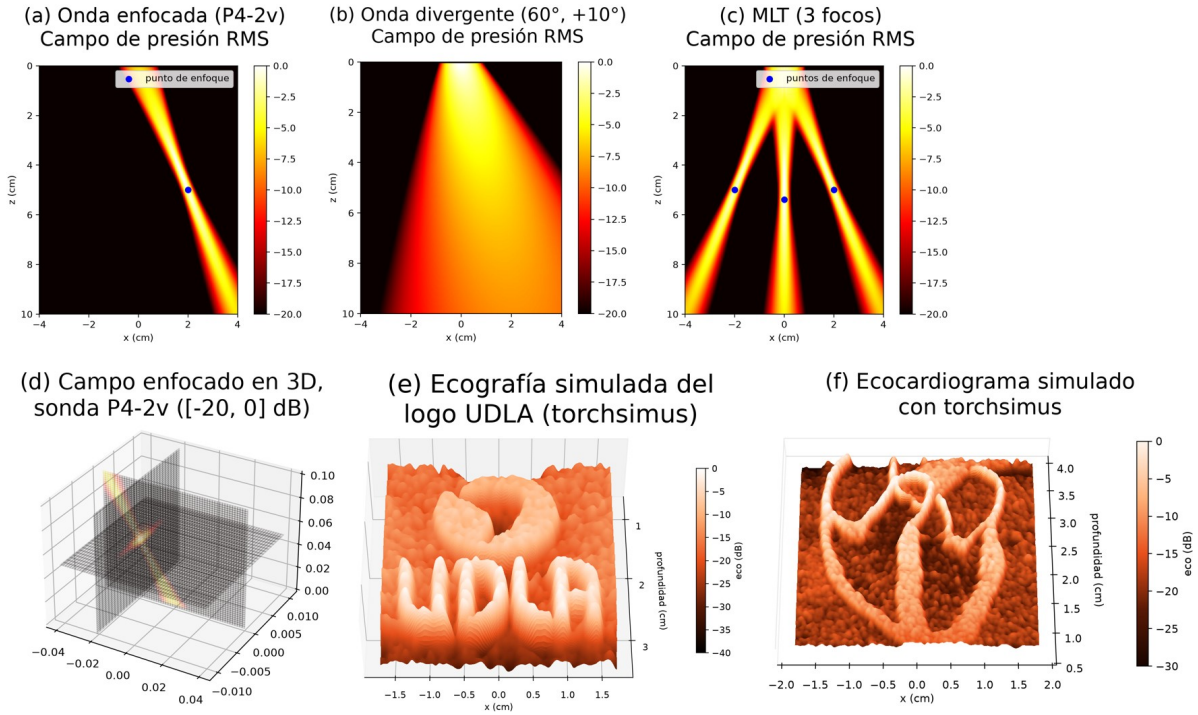
torchsimus calcula, con la física del sonido, lo que ocurriría en un examen real:

- **Campo de presión**: cómo se reparte la intensidad del sonido que emite la sonda (dónde queda el foco, cuánto se abre el haz).
- **Señales RF**: los ecos que recibiría cada elemento de la sonda frente a un tejido simulado (**fantoma**).
- **Imagen modo B, Doppler color y speckle tracking** a partir de esas señales, como lo hace un equipo de ultrasonido. También procesa señales medidas con equipos reales.
- **Películas** que muestran el avance de la onda de sonido.
- **Diseño de sondas**: puedes usar sondas comerciales o crear la tuya y ver cómo cambia el haz.

torchsimus es una reimplementación de SIMUS en PyTorch [4], un software de código abierto orientado a la investigación en imagenología médica. Ejecuta la misma física que SIMUS y agrega tres capacidades:

- **Corre en CPU o en tarjeta gráfica (GPU)**: en GPU, una ecografía cardíaca que tarda minutos se simula en segundos.
- **Procesa lotes de simulaciones** a la vez, útil para generar muchas imágenes de entrenamiento.
- **Es diferenciable**: calcula cómo cambia el resultado al cambiar la reflectividad del tejido, la posición de los dispersores o los retardos de emisión. Eso permite optimizarlos automáticamente.

Funciona con sondas lineales, convexas, matriciales, en anillo y personalizadas, y con emisiones enfocadas, planas, divergentes y de varios focos a la vez (MLT). Estos son algunos resultados simulados con torchsimus:

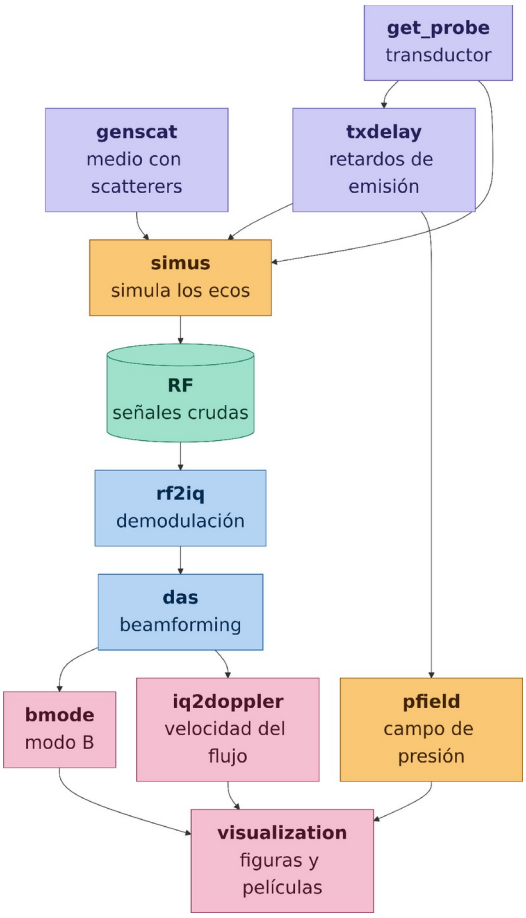


Campos de presión en 2-D (a–c) y en 3-D (d), una ecografía simulada sobre el logo de la UDLA (e) y un ecocardiograma simulado (f).

1.5 Flujo de una simulación

Una simulación sigue los mismos pasos que un equipo de ultrasonido real. Cada caja del diagrama es una función de torchsimus:

1. Se define la **sonda** (`get_probe`), el **tejido** con sus dispersores (`genscat`) y los **retardos de emisión** (`txdelay`).
2. `simus` calcula los ecos que recibe cada elemento: las **señales RF**.
3. La imagen se forma con la **demodulación** (`rf2iq`) y el **beamforming** (`das`).
4. Con eso se obtiene el **modo B** (`bmode`) o el **Doppler** (`iq2doppler`). En paralelo, `pfield` calcula el **campo de presión** de la emisión.



Flujo de una simulación con torchsimus.

1.6 Palabras que se usan en este instructivo

Término	Qué significa
Sonda, transductor o cabezal	El aparato que se apoya en la piel; emite y recibe el sonido
Elemento	Cada una de las pequeñas piezas que vibran dentro de la sonda
Pitch	Distancia entre los centros de dos elementos vecinos
Apertura	El ancho total de la parte de la sonda que está emitiendo
Retardos	Pequeñas diferencias de tiempo con que se dispara cada elemento; definen el tipo de onda
Foco	Punto donde se concentra el sonido
Apodización	Peso distinto para cada elemento (por ejemplo, menos en los bordes) para suavizar el haz
Atenuación	Pérdida de intensidad con la profundidad, medida en dB/cm/MHz
Campo de presión	Mapa de la intensidad del sonido emitido, en decibeles (0 dB = máximo)

Término	Qué significa
Dispersores y fantoma	Puntos que reflejan el sonido; un fantoma es un tejido simulado hecho con ellos
Señal RF e I/Q	Señal de eco que registra cada elemento; I/Q es la misma señal en una forma más fácil de procesar
Beamforming (DAS)	Algoritmo que combina las señales de todos los elementos para formar la imagen
Modo B	La imagen de ultrasonido en escala de grises (la ecografía)
Doppler color	Imagen de la velocidad del flujo sanguíneo
Speckle tracking	Seguimiento del movimiento del tejido entre imágenes

1.7 Qué necesitas

Solo una **cuenta de Google** para usar **Google Colab**, un servicio gratuito que ejecuta Python en el navegador. No necesitas instalar nada en tu computador ni descargar ningún archivo: torchsimus se instala directamente desde internet. Si prefieres trabajar en tu propio computador, también funciona con Python 3.9 o superior.

2. Instalar torchsimus en Google Colab

1. Entra a **colab.research.google.com** con tu cuenta de Google y crea un notebook nuevo: **Archivo → Nuevo notebook**.
2. Opcional pero recomendado: activa la tarjeta gráfica en **Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU (T4)**. Hazlo antes de correr nada, porque cambiar el tipo reinicia todo.
3. En la primera celda escribe la línea de abajo y córrela con el botón ▶️ (o Shift + Enter). Tarda alrededor de un minuto.

```
!pip install git+https://github.com/GianfrancoOneto/torchsimus.git
```

Cuando termine, ya puedes usar torchsimus en las celdas siguientes. La instalación dura mientras el notebook esté abierto: cada vez que vuelvas a abrirlo, corre de nuevo esa celda.

2.1 Actualizar a la última versión

```
!pip install --force-reinstall --no-deps
git+https://github.com/GianfrancoOneto/torchsimus.git
```

> **Importante:** después de actualizar, reinicia el entorno (**Entorno de ejecución → Reiniciar sesión**). Colab mantiene en memoria la versión anterior hasta reiniciar.

2.2 Archivos

Para subir un archivo (por ejemplo, las coordenadas de tu propio cabezal) o descargar uno que generaste (por ejemplo, una película GIF), usa el **panel de archivos** de Colab: el ícono de carpeta en la barra izquierda. Lo que subas queda en la carpeta `/content`, y puedes abrirlo desde el código solo con su nombre.

3. Primeros pasos: el sonido que emite la sonda

Copia cada bloque de código en una celda de Colab y córrelo. Los ejemplos de esta sección calculan el **campo de presión**: cómo se reparte el sonido que emite la sonda. En los gráficos, el color claro indica dónde el sonido es más intenso (0 dB es el máximo).

3.1 Celda de preparación

Córrela siempre primero. Carga las funciones y elige la GPU si está disponible:

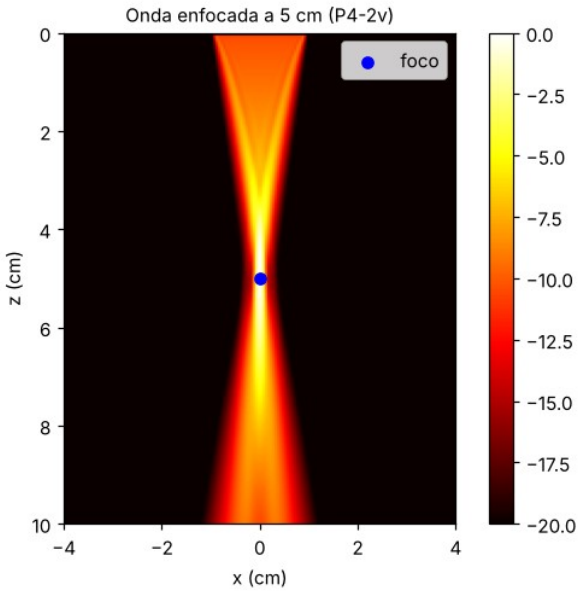
```
import math, numpy as np, torch, matplotlib.pyplot as plt
from torchsimus.probes import get_probe
from torchsimus.delays import txdelay_focus, txdelay_plane, txdelay_diverging
from torchsimus.field import pfield, pfield3
from torchsimus.visualization import show_field, show_polar_field

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"   # GPU si hay, si no CPU
deg = math.pi / 180                                       # para escribir ángulos en grados
print("Usando:", DEVICE)
```

3.2 Onda enfocada

Los elementos de la sonda se disparan con retardos para que el sonido se concentre en un punto (el foco), aquí a 5 cm de profundidad:

```
sonda = get_probe("P4-2v").to(DEVICE)            # sonda de 64 elementos, 2.72 MHz
d = txdelay_focus(sonda, 0.0, 5e-2)              # foco en x = 0, z = 5 cm (en metros)
x, z = np.meshgrid(np.linspace(-4e-2, 4e-2, 160), np.linspace(0, 10e-2, 200))
P = pfield(sonda, x, None, z, d)                 # intensidad del sonido en cada punto
show_field(P, x, z, "Onda enfocada a 5 cm",
           points=(0.0, 5e-2), points_label="foco")
plt.show()
```

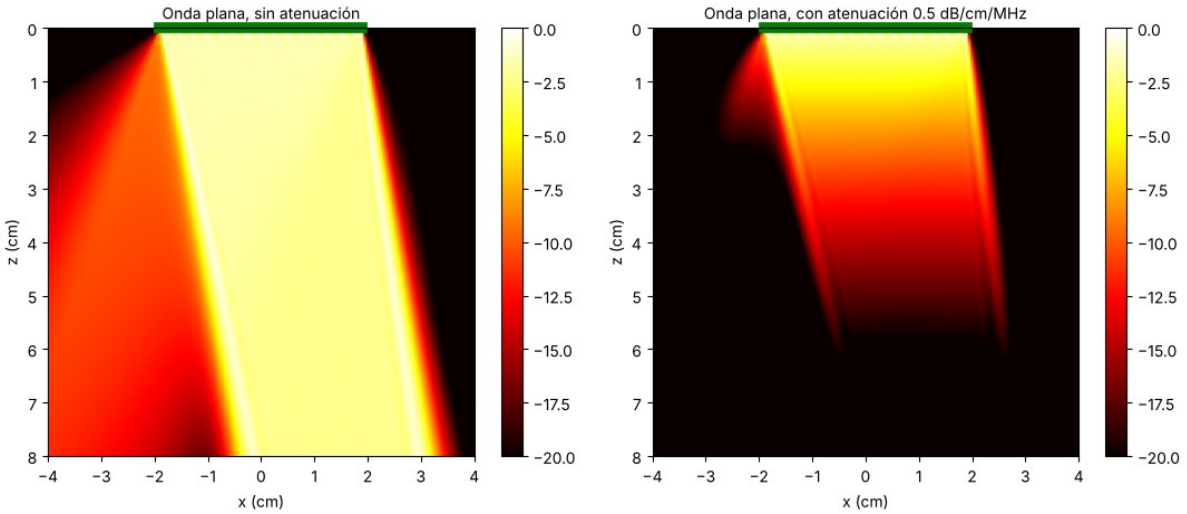


Onda enfocada: el sonido se concentra a 5 cm de profundidad.

3.3 Onda plana, con y sin atenuación

En una onda plana todos los elementos disparan casi a la vez y el sonido avanza como un frente recto. El segundo gráfico muestra cómo el tejido debilita el sonido con la profundidad (atenuación):

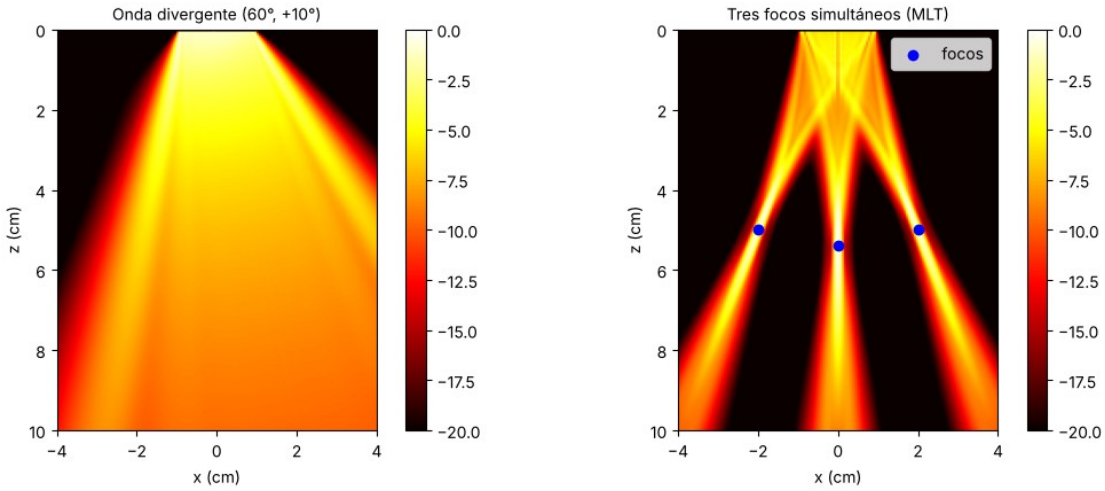
```
sonda = get_probe("L11-5v").to(DEVICE)           # sonda lineal de 128 elementos, 7.6 MHz
d = txdelay_plane(sonda, 10 * deg)               # onda plana inclinada 10°
x, z = np.meshgrid(np.linspace(-4e-2, 4e-2, 150), np.linspace(0, 8e-2, 150))
xe = sonda.pose().centers.cpu().numpy()          # posición de los elementos
fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
for a, att, titulo in [(ax[0], 0.0, "sin atenuación"), (ax[1], 0.5, "con atenuación 0.5
dB/cm/MHz")]:
    P = pfield(sonda, x, None, z, d, attenuation=att)
    show_field(P, x, z, f"Onda plana, {titulo}", ax=a)
    a.plot(xe[:, 0] * 1e2, xe[:, 2] * 1e2, c="g", linewidth=7, clip_on=False)   # la sonda
plt.tight_layout(); plt.show()
```



Onda plana inclinada 10° sin y con atenuación. La línea verde marca la sonda.

3.4 Onda divergente y varios focos a la vez

```
sonda = get_probe("P4-2v").to(DEVICE)
x, z = np.meshgrid(np.linspace(-4e-2, 4e-2, 160), np.linspace(0, 10e-2, 200))
d_div = txdelay_diverging(sonda, 10 * deg, 60 * deg)   # onda que se abre 60°, inclinada 10°
xf = np.array([-2e-2, 0.0, 2e-2]); zf = np.array([5e-2, 5.4e-2, 5e-2])
d_mlt = txdelay_focus(sonda, xf, zf)                   # tres focos al mismo tiempo (MLT)
fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
show_field(pfield(sonda, x, None, z, d_div), x, z, "Onda divergente (60°, +10°)", ax=ax[0])
show_field(pfield(sonda, x, None, z, d_mlt), x, z, "Tres focos simultáneos (MLT)", ax=ax[1],
           points=(xf, zf), points_label="focos")
plt.tight_layout(); plt.show()
```

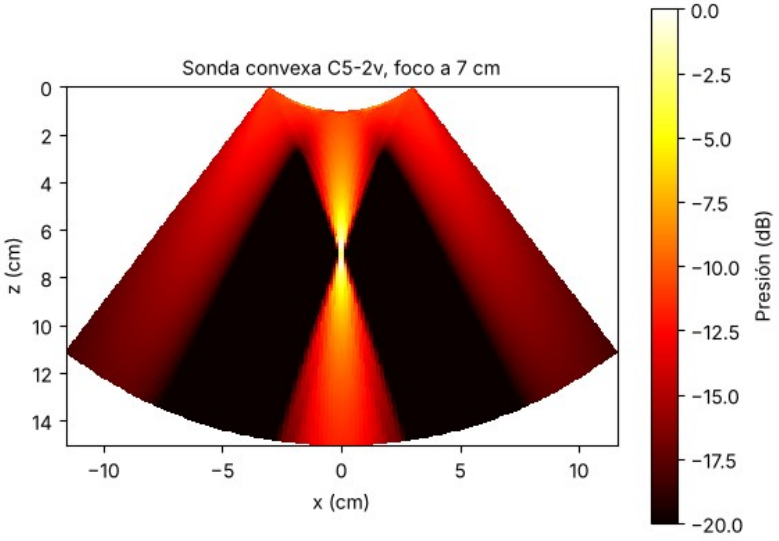


Onda divergente (izquierda) y emisión con tres focos simultáneos (derecha).

3.5 Sonda convexa (abdominal)

Las sondas convexas forman una imagen en forma de abanico, por eso el campo se calcula en una grilla de ese tipo:

```
from torchsimus.processing.grids import impolgrid
sonda = get_probe("C5-2v").to(DEVICE)            # sonda convexa de 128 elementos
d = txdelay_focus(sonda, 0.0, 7e-2)              # foco a 7 cm
x, z = impolgrid(sonda, (128, 128), 15e-2)       # grilla en abanico hasta 15 cm
P = pfield(sonda, x, None, z, d)
show_polar_field(P, x, z, "Sonda convexa C5-2v, foco a 7 cm"); plt.show()
```



Campo de una sonda convexa enfocada a 7 cm.

3.6 Cambiar de sonda

En todos los ejemplos puedes reemplazar el nombre de la sonda por cualquiera de las 10 incluidas (lista en la sección 6.2). Con `get_probe("L11-5v", 24)` se usa solo un grupo de 24 elementos de la misma sonda.

4. Simular una imagen de ultrasonido

4.1 Imagen modo B de un tejido con un quiste

Este ejemplo recorre todo el proceso de un equipo de ultrasonido: crea un tejido simulado (fantoma) con un quiste, calcula los ecos que recibe cada elemento, forma la imagen y la muestra en escala de grises.

```
from torchsimus.phantoms import genscat
from torchsimus.simulation import simus
from torchsimus.processing.rf2iq import rf2iq
from torchsimus.processing.das import das
from torchsimus.processing.doppler import bmode

sonda = get_probe("L11-5v").to(DEVICE)
d = txdelay_plane(sonda, 0.0)                    # onda plana a 0°

np.random.seed(0)
xs, ys, zs, RC = genscat([2e-2, 3e-2], 1540 / sonda.fc / 2)  # tejido de 2 x 3 cm
zs = zs + 0.5e-2                                 # el tejido va de 0.5 a 3.5 cm
RC = RC.clone()
RC[(xs**2 + (zs - 2e-2)**2) < (4e-3)**2] = 0     # quiste de 4 mm de radio a 2 cm

fs = 4 * sonda.fc                                # frecuencia de muestreo
RF = simus(sonda, xs, None, zs, RC, d, fs=fs)    # ecos de cada elemento
IQ = rf2iq(RF, fs, sonda.fc)                     # demodulación
x, z = np.meshgrid(np.linspace(-1e-2, 1e-2, 200), np.linspace(0.5e-2, 3.5e-2, 300))
IQb = das(IQ.to(DEVICE), x, z, d, sonda, fs)     # beamforming
img = bmode(IQb, 40).cpu().numpy()               # modo B, 40 dB de rango
plt.imshow(img, cmap="gray", extent=[-1, 1, 3.5, 0.5])
plt.xlabel("x (cm)"); plt.ylabel("z (cm)"); plt.show()
```

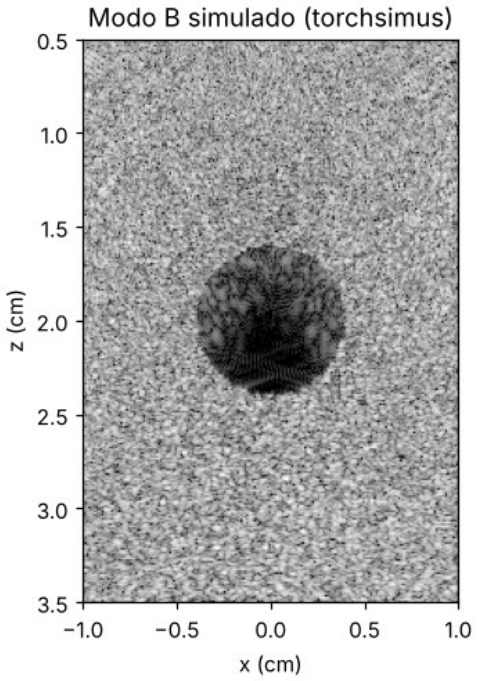


Imagen modo B simulada: el quiste aparece como un círculo oscuro a 2 cm.

> **Nota:** este ejemplo simula unos 23 000 puntos de eco y tarda alrededor de 2,5 minutos en CPU; con GPU, unos segundos. Para una prueba rápida, cambia `1540 / sonda.fc / 2` por `1540 / sonda.fc * 2` (menos puntos).

4.2 Película de la propagación del sonido

Genera una animación (GIF) de la onda avanzando desde la sonda y la muestra en Colab:

```
from torchsimus.movie import mkmovie
from IPython.display import Image
sonda = get_probe("P4-2v").to(DEVICE)
d = txdelay_focus(sonda, 0.0, 2e-2)              # onda enfocada a 2 cm
F, info = mkmovie(sonda, d, movie=[3, 6, 100], gif_path="propagacion.gif")
Image("propagacion.gif")
```

El archivo `propagacion.gif` queda en el panel de archivos, desde donde puedes descargarlo. Para ver el efecto de la atenuación, agrega `attenuation=0.5` dentro de `mkmovie`.

4.3 Procesar datos reales: un disco que gira

torchsimus también procesa señales medidas con un equipo real. Este ejemplo usa una adquisición de un disco que gira (archivo `PWI_disk.mat`, 2 MB): un lado del disco se acerca a la sonda y el otro se aleja. Con esos datos se obtienen la imagen modo B, el **Doppler color** (velocidad) y el **speckle tracking** (movimiento).

Descarga del archivo: https://github.com/GianfrancoOneto/torchsimus/raw/main/Datos/PWI_disk.mat

En Colab no hace falta descargarlo a mano: esta línea lo baja directo a la carpeta del notebook.

```
!wget -q https://github.com/GianfrancoOneto/torchsimus/raw/main/Datos/PWI_disk.mat
```

Luego corre (después de la celda de preparación 3.1):

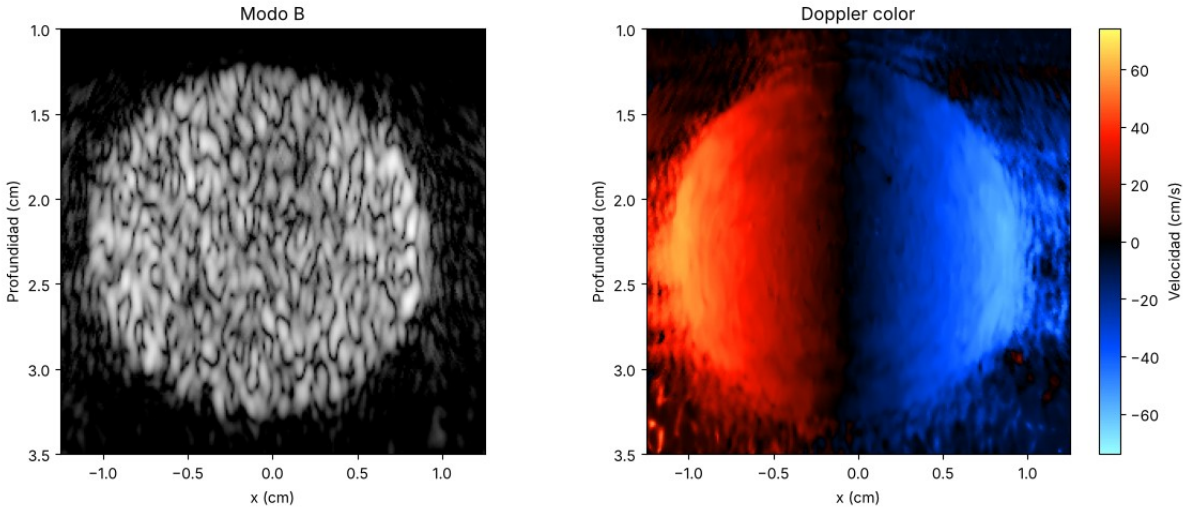
```
import scipy.io
from torchsimus.probes import make_transducer
from torchsimus.processing.rf2iq import rf2iq
from torchsimus.processing.das import das
from torchsimus.processing.doppler import bmode, iq2doppler, nyquist_velocity
from torchsimus.processing.sptrack import sptrack
from torchsimus.visualization import doppler_cmap

m = scipy.io.loadmat("PWI_disk.mat", struct_as_record=False, squeeze_me=True)
RF = torch.as_tensor(m["RF"].astype(np.float64))          # ecos: [muestras, elementos,
emisiones]
p = m["param"]                                            # parámetros de la adquisición
fc, fs, c = float(p.fc), float(p.fs), float(getattr(p, "c", 1540.0))
prf = float(p.PRF) if hasattr(p, "PRF") else 1 / float(p.PRP)
t0 = float(np.ravel(getattr(p, "t0", 0.0))[0])
ancho = float(p.width) if hasattr(p, "width") else float(p.pitch) - float(getattr(p, "kerf",
0.0))
banda = float(getattr(p, "bandwidth", 75)) / 100
sonda = make_transducer(fc, float(p.pitch), ancho, int(p.Nelements), banda,
                        radius=float(getattr(p, "radius", np.inf))).to(DEVICE)
retardos = torch.as_tensor(np.ravel(p.TXdelay).astype(np.float64))

paso = 1e-4                                               # píxel de 0.1 mm
x, z = np.meshgrid(np.arange(-1.25e-2, 1.25e-2 + paso, paso),
                   np.arange(1e-2, 3.5e-2 + paso - 1e-6, paso))
IQ = rf2iq(RF, fs, fc, banda, t0=t0)                      # demodulación
```

```
IQb = das(IQ.to(DEVICE), x, z, retardos, sonda, fs, fc, c=c, t0=t0,
          fnumber=None, method="nearest").cpu()            # beamforming: [z, x, emisiones]
imagen = bmode(IQb, 30).numpy()                            # modo B
vel, var = [t.numpy() for t in iq2doppler(IQb, fc, prf, c)]   # Doppler color (m/s)
VN = nyquist_velocity(fc, prf, c)                          # velocidad máxima medible

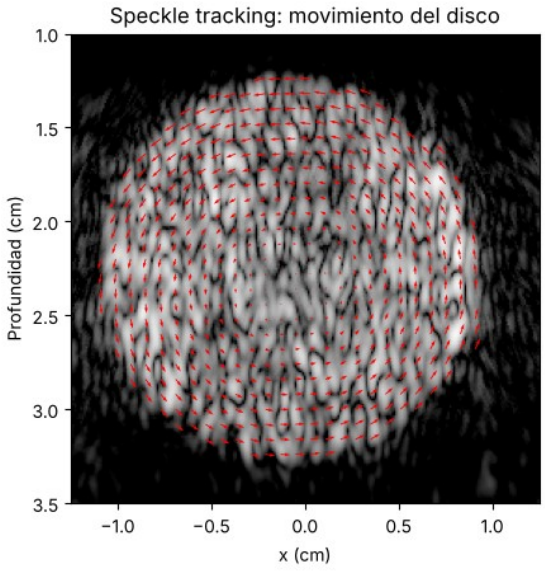
ext = [-1.25, 1.25, 3.5, 1]
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].imshow(imagen[:, :, 0], cmap="gray", extent=ext); ax[0].set_title("Modo B")
im = ax[1].imshow(vel * 100, cmap=doppler_cmap(), vmin=-VN * 100, vmax=VN * 100, extent=ext)
plt.colorbar(im, ax=ax[1]).set_label("Velocidad (cm/s)"); ax[1].set_title("Doppler color")
for a in ax: a.set_xlabel("x (cm)"); a.set_ylabel("Profundidad (cm)")
plt.tight_layout(); plt.show()
```



Modo B y Doppler color del disco: en rojo la mitad que se acerca a la sonda y en azul la que se aleja.

Para ver el movimiento con flechas (speckle tracking), agrega esta celda:

```
Di, Dj, ix, iz = sptrack(imagen, [[32, 32], [16, 16]], iminc=4, roi=np.median(imagen, 2) >
64)
plt.imshow(imagen[:, :, 0], cmap="gray", extent=ext)
plt.quiver(ix * paso * 100 - 1.25, iz * paso * 100 + 1, -Di * paso * 100, Dj * paso * 100,
color="r")
plt.title("Speckle tracking: movimiento del disco"); plt.show()
```



Speckle tracking: las flechas muestran que el disco gira.

5. Diseñar tu propio cabezal

Además de las sondas comerciales, puedes simular un cabezal con cualquier forma: basta con saber dónde está cada elemento y hacia dónde mira.

5.1 Dos funciones de ayuda

Copia esta celda una vez. `crear_cabezal` arma la sonda a partir de las posiciones de sus elementos, y `onda_plana` calcula los retardos de una onda plana para cualquier forma:

```
from torchsimus.geometry.custom import CustomArray
from torchsimus.elements.rectangular import RectangularElement
from torchsimus.transducer import Transducer

def crear_cabezal(centros, normales, ancho, alto, fc=3e6, banda=0.7):
    """Sonda a partir de las posiciones (m) y normales de sus elementos."""
    n = torch.nn.functional.normalize(torch.as_tensor(normales, dtype=torch.float64), dim=-1)
    ref = torch.tensor([1.0, 0.0, 0.0], dtype=torch.float64).expand_as(n)
    u = torch.nn.functional.normalize(ref - (ref * n).sum(-1, keepdim=True) * n, dim=-1)
    v = torch.cross(n, u, dim=-1)
    el = RectangularElement(ancho); el.height = alto
    sonda = Transducer(CustomArray(torch.as_tensor(centros, dtype=torch.float64), u, v, n),
el,
                       center_frequency=fc, bandwidth=banda)
    sonda.width, sonda.height = ancho, alto
    return sonda

def onda_plana(sonda, angulo, c=1540.0):
    """Retardos de una onda plana inclinada `angulo` (rad) para cualquier sonda."""
    centros = sonda.pose().centers
    k = torch.tensor([math.sin(angulo), 0.0, math.cos(angulo)], dtype=centros.dtype,
device=centros.device)
    t = centros @ k / c
    return (t - t.min())[None, :]
```

5.2 Usar uno de los 35 cabezales de ejemplo

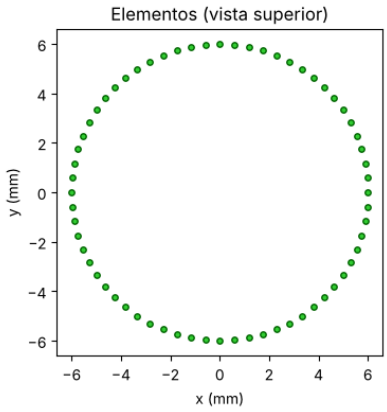
torchsimus trae 35 cabezales listos, 5 de cada tipo: lineal, convexo, matricial, circular (un solo elemento), anillo, espiral y otras formas (cóncavo, en V, cruz, doble anillo y matriz dispersa). Este ejemplo carga un anillo de 64 elementos, lo dibuja y calcula su campo:

```
from torchsimus import cabezales
print(cabezales.listar("anillo"))                # nombres disponibles de ese tipo

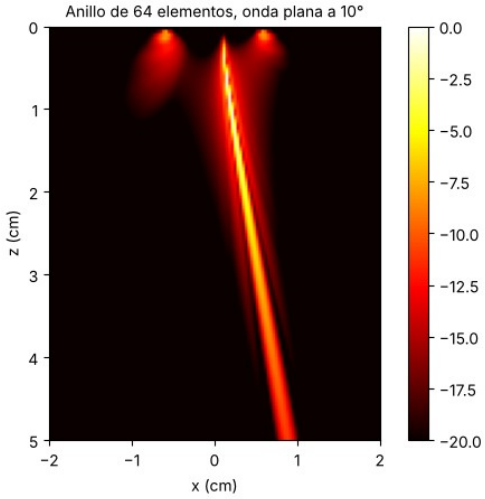
centros, normales = cabezales.cargar("anillo_1_64el_R6mm")
mi_sonda = crear_cabezal(centros, normales, ancho=0.25e-3, alto=0.25e-3).to(DEVICE)

c = mi_sonda.pose().centers.cpu().numpy() * 1e3  # en mm
plt.scatter(c[:, 0], c[:, 1], s=15, c="limegreen", edgecolors="darkgreen")
plt.gca().set_aspect("equal"); plt.title("Elementos (vista superior)"); plt.show()

x, z = np.meshgrid(np.linspace(-2e-2, 2e-2, 120), np.linspace(0, 5e-2, 120))
P = pfield3(mi_sonda, x, np.zeros_like(x), z, onda_plana(mi_sonda, 10 * deg))
show_field(P, x, z, "Anillo de 64 elementos, onda plana a 10°"); plt.show()
```



Elementos del anillo vistos desde arriba.



Campo del anillo: forma un haz muy delgado.

Para ver todos los nombres usa `cabezales.listar()`. Cada nombre indica el tipo, el número de elementos y su medida principal, por ejemplo `convexo_1_128el_R49.57mm`.

5.3 Usar tus propias coordenadas

Prepara un archivo CSV (puede ser una planilla de Excel guardada como CSV) con una fila por elemento y estas columnas, en milímetros:

```
x,y,z,nx,ny,nz
-4.5000,0.0000,0.0000,0.148340,0.000000,0.988936
-4.2097,0.0000,0.0435,0.148340,0.000000,0.988936
...
```

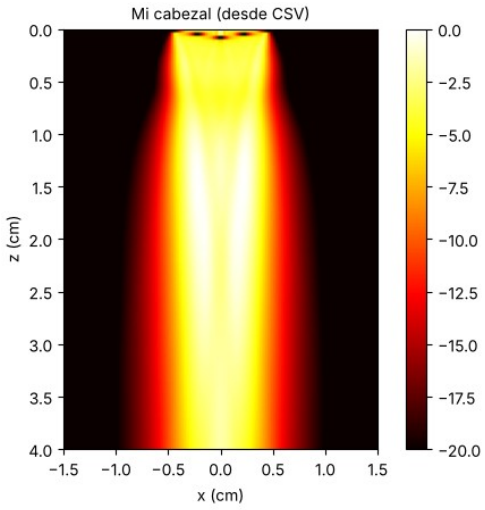
- `x`, `y`, `z`: posición del centro de cada elemento, en mm. El eje z apunta hacia el cuerpo.
- `nx`, `ny`, `nz`: hacia dónde mira cada elemento (opcional). Si no las pones, todos miran hacia el cuerpo (+z).

Súbelo con el panel de archivos y córrelo así:

```
import pandas as pd
tabla = pd.read_csv("mi_cabezal.csv")
```

```
centros = tabla[["x", "y", "z"]].values * 1e-3           # mm -> m
normales = tabla[["nx", "ny", "nz"]].values if "nx" in tabla else np.tile([0.0, 0.0, 1.0],
(len(tabla), 1))
mi_sonda = crear_cabezal(centros, normales, ancho=0.29e-3, alto=5e-3).to(DEVICE)

x, z = np.meshgrid(np.linspace(-1.5e-2, 1.5e-2, 120), np.linspace(0, 4e-2, 120))
P = pfield3(mi_sonda, x, np.zeros_like(x), z, onda_plana(mi_sonda, 0.0))
show_field(P, x, z, "Mi cabezal"); plt.show()
```



Campo de un cabezal en forma de V cargado desde un CSV.

> **Nota:** `ancho` y `alto` son las medidas de cada elemento en metros. Para que los elementos queden pegados sin superponerse, usa como ancho la distancia entre centros vecinos.

6. Referencia rápida

6.1 Módulos y funciones principales

Módulo	Funciones	Para qué sirve	En MUST
`probes`	`get_probe`, `make_transducer`, `matrix_probe`	Crear sondas comerciales o propias	getparam
`delays`	`txdelay_focus`, `txdelay_plane`, `txdelay_diverging`, `txdelay3_focus`, `txdelay3_plane`, `txdelay3_diverging`, `embed_subaperture`	Leyes de retardo de transmisión 2-D y 3-D	txdelay, txdelay3
`field`	`pfield`, `pfield3`	Campo de presión RMS (1-D con elevación; arreglos planos)	pfield, pfield3
`simulation`	`simus`, `simus3`	Señales RF con la física completa	simus, simus3
`simulator`	`Simus`	Modelo 2-D rápido con batch (nn.Module)	—
`movie`	`mkmovie`	Película de la propagación (GIF)	mkmovie
`phantoms`	`genscat`	Fantomas de scatterers 2-D y 3-D	genscat
`pulse`	`getpulse`	Pulso de transmisión en el tiempo	getpulse
`processing`	`rf2iq`, `das`, `das3`, `tgc`, `bmode`, `iq2doppler`, `sptrack`, `impolgrid`	Demodulación, beamforming, ganancia, modo B, Doppler, speckle tracking, grilla polar	rf2iq, das, tgc, bmode, iq2doppler, sptrack, impolgrid
`visualization`	`show_field`, `show_polar_field`, `slice_plot`, `single_slice_plot`, `doppler_cmap`	Gráficos de campos e imágenes	—
`cabezales`	`listar`, `cargar`, `radio`, `ruta`	Cabezales de ejemplo en CSV	—

Módulo	Funciones	Para qué sirve	En MUST
`geometry`	`LinearArray`, `ConvexArray`, `MatrixArray`, `RingArray`, `CustomArray`, `CircularSingleElement`	Posición y orientación de los elementos	—

6.2 Sondas incluidas

Sonda	Tipo	Elementos	fc (MHz)	Pitch (mm)
P4-2v	Phased array	64	2.72	0.300
L11-5v	Lineal	128	7.60	0.300
L12-3v	Lineal	192	7.54	0.200
C5-2v	Convexo (R 49.57 mm)	128	3.57	0.508
PA4-2/20	Phased array	64	2.50	0.300
L9-4/38	Lineal	128	5.00	0.305
LA530	Lineal	192	3.00	0.245
L14-5/38	Lineal	128	7.20	0.305
L14-5W/60	Lineal	128	7.50	0.472
P6-3	Phased array	64	4.50	0.218

`get_probe("L11-5v", 24)` crea una subapertura de 24 elementos de la misma sonda.

6.3 Unidades y convenciones

- Todas las posiciones en **metros**, tiempos en **segundos**, frecuencias en **Hz** y ángulos en **radianes**. En los ejemplos se usa `deg = math.pi / 180` para escribir grados (`10 * deg`).
- La atenuación se da en **dB/cm/MHz** (por ejemplo `attenuation=0.5`).
- Los retardos tienen forma `[E, N]`: una fila por emisión y una columna por elemento. Varias filas a la vez = emisiones simultáneas (MLT).
- Un retardo `NaN` significa elemento apagado.
- El eje z apunta hacia el tejido: z positivo es profundidad.
- Cada elemento tiene tres direcciones: `u` (a lo largo del ancho), `v` (a lo largo del alto) y `n` (la normal, hacia donde emite).

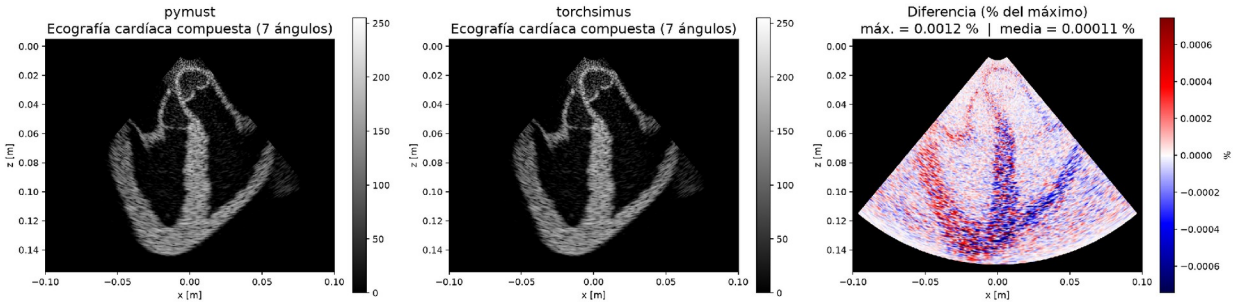
7. Confiabilidad y rapidez

torchsimus entrega **los mismos resultados que pymust**, la versión en Python del simulador de referencia, y suma lo que pymust no tiene:

	pymust	torchsimus
Física de SIMUS	Sí	Sí
Cálculo en GPU	No	Sí
Diferenciable (gradientes)	No	Sí
Lotes de simulaciones	No	Sí
Uso con redes neuronales	No	Sí

7.1 Comparación con pymust

La misma ecografía cardíaca compuesta (sonda P4-2v, 7 ondas divergentes, unas 28 000 fuentes de eco) simulada con ambos programas da imágenes indistinguibles: la diferencia máxima es de 0,0012 % de la intensidad máxima.



Ecografía cardíaca simulada con pymust (izquierda) y con torchsimus (centro), y su diferencia en % del máximo (derecha).

Comparación con pymust	Diferencia
Campo de presión (12 escenas)	$\leq 9 \times 10^{-6}$
Campo 3-D, matriz 32×32	9×10^{-7}
Señales RF, corazón	4×10^{-5}
Modo B compuesto	0,0004 dB
Doppler, datos reales	0,000 cm/s

Diferencias expresadas como error relativo, salvo cuando se indica la unidad. Además, el paquete aprueba **96 pruebas automáticas**.

7.2 Tiempos de cálculo

Simulación	pymust (CPU)	torchsimus (CPU)	torchsimus (GPU)
Ecografía cardíaca (RF)	395 s	257 s	**17,2 s**

Simulación	pymust (CPU)	torchsimus (CPU)	torchsimus (GPU)
Película (`mkmovie`)	75 s	41 s	**0,7 s**
Campo 3-D, matriz 32×32	–	349 s	**7,2 s**
Onda plana con atenuación	–	105 s	**2,5 s**
Subaperturas L11-5v	–	64 s	**1,9 s**

CPU: 1 hilo. GPU: NVIDIA Tesla T4 (la que ofrece Google Colab). –: no medido.

8. Problemas frecuentes

Mensaje o síntoma	Causa	Solución
`No existe el cabezal …`	La versión instalada es antigua	Actualiza con `!pip install --force-reinstall --no-deps git+https://github.com/GianfrancoOneto/torchsimus.git` y reinicia el entorno
`ModuleNotFoundError: No module named 'torchsimus'`	No se corrió la instalación en esta sesión	Corre la celda `!pip install …` y después la celda de preparación (3.1)
`NameError: name 'DEVICE' is not defined` (u otro nombre)	No se corrió la celda de preparación	Corre la celda de preparación (3.1) antes de los ejemplos
`Expected all tensors to be on the same device`	Se mezclaron cálculos en CPU y GPU	Mueve la sonda con `.to(DEVICE)` antes de simular, como en los ejemplos
`AttributeError` o una función que no existe después de actualizar	Colab guarda en memoria la versión anterior	Reinicia la sesión (Entorno de ejecución → Reiniciar sesión) y vuelve a correr la preparación
Una simulación tarda mucho	Muchos puntos, elementos o una grilla grande, en CPU	Activa la GPU (sección 2) o usa grillas y fantomas más chicos
`!wget` da error 404 o `PWI_disk.mat` no se encuentra	El archivo no se descargó	Revisa la conexión y vuelve a correr `!wget`, o descárgalo desde el link de la sección 4.3 y súbelo con el panel de archivos
Se borró todo lo que había corrido	Colab reinicia la sesión tras un tiempo sin uso o al cambiar a GPU	Vuelve a correr la instalación y la celda de preparación

9. Anexo: el modelo físico

Esta sección es para quien quiera saber cómo calcula torchsimus. No hace falta para usarlo.

El modelo trabaja en el dominio de la frecuencia. Para un dispersor s ubicado a una distancia r_{sn} del elemento n, la **función de Green** describe cómo llega el sonido de uno a otro:

$$G_{sn}(f) = e^{i \cdot k \cdot rsn} / \sqrt{r_{sn}} \cdot D_n(\theta, f), \qquad k = 2\pi f / c$$

donde D_n es la **directividad** del elemento (cuánto emite en cada dirección), θ el ángulo respecto de su normal, f la frecuencia y c la velocidad del sonido (1540 m/s). Si cada elemento emisor m tiene apodización w_m y retardo τ_m, el espectro **pulso-eco** que recibe el elemento n es:

$$Y_n(f) = Q(f) \cdot \Sigma_s \, G_{sn} \, R_s \cdot \Sigma_m \, G_{sm} \, w_m \, e^{i \cdot 2\pi \cdot f \cdot \tau m}$$

donde R_s es la **reflectividad** del dispersor y Q(f) el espectro pulso-eco de la sonda. Las señales RF se obtienen con una transformada de Fourier inversa. Como cada paso es una operación con tensores de PyTorch, los gradientes se calculan automáticamente.

10. Referencias

[1] Damien Garcia. SIMUS: an open-source simulator for medical ultrasound imaging. Part I: theory & examples. Computer Methods and Programs in Biomedicine, 218:106726, 2022.

[2] Damien Garcia. MUST: Matlab UltraSound Toolbox. https://www.biomecardio.com/MUST.

[3] Amanda Cigier, François Varray y Damien Garcia. SIMUS: an open-source simulator for medical ultrasound imaging. Part II: comparison with four simulators. Computer Methods and Programs in Biomedicine, 220:106774, 2022.

[4] Adam Paszke et al. PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, volumen 32, 2019.